ConsultAI 360 : PostgreSQL, qualité des données, entrepôt de données et analyse business

Cas d'étude NexaRetail
Rapport technique et business
Auteur : Can Pekgoz

Synthèse exécutive

ConsultAI 360 est un projet de conseil en données et en intelligence artificielle construit autour d'une entreprise fictive, NexaRetail.

La phase 4 met en place, sous PostgreSQL, une chaîne de traitement complète : import des données brutes, nettoyage, entrepôt de données dimensionnel, puis calcul d'indicateurs business.

Trois variantes volontairement dégradées du jeu de données (LOW, MEDIUM et CRITICAL) servent à mesurer l'effet de la qualité des données sur les indicateurs. Les résultats portent sur une simulation et ne décrivent pas une entreprise réelle.

Sur 120 000 commandes, le chiffre d'affaires de référence s'élève à 72 056 406,60 €. À volume de commandes constant, le chiffre d'affaires mesuré recule dans chaque variante :

Scénario	CA mesuré	Écart avec la référence	Écart relatif
ORIGINAL (référence)	72 056 406,60 €	—	—
LOW	71 327 494,22 €	−728 912,38 €	−1,01 %
MEDIUM	68 448 127,94 €	−3 608 278,66 €	−5,01 %
CRITICAL	64 901 512,00 €	−7 154 894,60 €	−9,93 %

L'écart maximal atteint 7 154 894,60 €. Il s'agit d'un biais de mesure induit par la simulation, et non d'une perte financière réelle.

1. Objectifs et périmètre

Le projet poursuit six objectifs : importer des données de ventes et de relation client, simuler puis auditer des corruptions de données, construire un entrepôt analytique, mesurer l'effet de ces corruptions sur les indicateurs clés, vérifier les résultats, et enfin formuler des recommandations en matière de Business Intelligence et de gouvernance des données.

L'environnement technique repose sur PostgreSQL 18.6, SQL, Python 3.12 (bibliothèque psycopg), DBeaver, VS Code et Git/GitHub. Le projet dispose par ailleurs d'une source textuelle indépendante, destinée à de futures expérimentations de traitement du langage naturel.

2. Architecture et lignage

Les données traversent quatre couches successives, chacune ayant un rôle précis.

Couche	Rôle	Contenu
RAW	Conservation de la donnée source	4 107 649 lignes
CLEAN	Données typées et contrôlées	3 019 182 lignes
DW	Modèle dimensionnel d'analyse	1 555 362 lignes de faits
Analytics	Requêtes d'indicateurs et d'optimisation	Vues et requêtes SQL

Un schéma supplémentaire, recovered, existe pour des essais de restauration qui n'ont pas été exécutés.

2.1 Couche RAW

La couche RAW stocke les valeurs métier au format TEXT, accompagnées de quatre champs de traçabilité : raw_id, scenario, source_file et source_row_number.
Ce choix conserve la preuve d'origine, y compris les anomalies de format, et permet de réexaminer chaque transformation. Le volume se répartit ainsi : ORIGINAL 1 006 394 lignes, LOW 1 011 524, MEDIUM 1 032 042 et CRITICAL 1 057 689.

2.2 Couche CLEAN

La couche CLEAN comprend dix tables typées : customers, products, stores, employees, orders, order_items, payments, shipments, inventory et complaints.
Les fichiers du dossier data/processed_v2 sont audités avant chargement. Chaque scénario contient 1 006 394 lignes, réparties comme suit :

Table	Lignes par scénario
Clients	120 000
Produits	8 000
Magasins	35
Employés	450
Commandes	120 000
Lignes de commande	272 953
Paiements	125 501
Expéditions	70 567
Stocks	280 000
Réclamations	8 888

Le nettoyage est contrôlé, mais il ne prétend pas restaurer des identifiants métier introuvables.

2.3 Entrepôt de données (DW)

L'entrepôt comporte quatre dimensions (clients, produits, magasins, dates) et trois tables de faits (commandes, ventes, paiements).
Chaque dimension métier contient un membre « Inconnu » par scénario, qui accueille les références sans correspondance. Le champ scenario doit figurer dans chaque jointure métier.

Sans lui, un identifiant réutilisé dans plusieurs variantes multiplierait artificiellement les résultats. Des clés techniques (clés de substitution) évitent par ailleurs de dépendre directement des identifiants de la source.

La granularité de chaque table de faits est la suivante : fact_orders contient une commande par scénario, fact_sales une ligne de commande par scénario, et fact_payments un paiement par scénario.

Les montants des commandes ne sont pas additionnés après jointure avec les lignes de vente ou les paiements, car ces relations peuvent être de type un-à-plusieurs.
Les montants de chaque table de faits sont donc conservés et analysés séparément.
L'entrepôt contient 360 000 commandes, 818 859 lignes de vente et 376 503 paiements (soit 1 555 362 lignes de faits), ainsi que 360 003 membres clients, 24 003 produits, 108 magasins et 730 dates.

3. Anomalies et stratégie de qualité

3.1 Anomalies mesurées

Trois catégories de références invalides ont été identifiées, dans des proportions croissantes selon le scénario.

Anomalie →	LOW	MEDIUM	CRITICAL	Total
Commandes avec client inconnu →	1 200	6 000	12 000	19 200
Lignes de vente avec produit inconnu →	2 730	13 648	27 295	43 673
Paiements sans commande correspondante →	1 255	6 275	12 550	20 080

Ces trois catégories totalisent 82 953 lignes affectées.
Ce chiffre compte des lignes, et non des transactions uniques : une même transaction peut apparaître dans plusieurs catégories.

Les dates de commande absentes suivent la même progression :
1 200 commandes en LOW (1 %), 6 000 en MEDIUM (5 %) et 12 000 en CRITICAL (10 %), soit 19 200 au total. Ces dates manquent déjà dans la couche CLEAN : l'entrepôt ne les a pas perdues. Les regroupements mensuels excluent donc ces commandes, faute de catégorie explicite « Date inconnue ».

3.2 Principes de traitement

Les ventes sont conservées grâce aux jointures externes (LEFT JOIN) et aux dimensions « Inconnu ». Les paiements sans commande restent dans fact_payments avec une clé order_key à NULL, car les supprimer fausserait les montants. Une transaction non attribuable n'est pas une transaction perdue.

3.3 Restauration des identifiants

La restauration des identifiants a été étudiée en confrontant les variantes à la couche RAW ORIGINAL. Des correspondances sur source_row_number et sur les identifiants étaient disponibles, mais 1 628 montants de paiement différaient aussi entre les variantes et l'original.
Le chantier a été volontairement différé afin d'éviter une correction non gouvernée.
Le schéma recovered contient une copie expérimentale des dix tables CLEAN (3 019 182 lignes) et une table d'audit vide ; aucune restauration n'a été appliquée.
En production, il faudrait au préalable valider la source de vérité, documenter les règles, conserver les valeurs avant et après correction (old_value / new_value), tester les collisions et obtenir une validation métier.

4. Résultats business

4.1 Chiffre d'affaires

Le chiffre d'affaires de référence (ORIGINAL) est de 72 056 406,60 € pour 120 000 commandes. Le tableau de la synthèse détaille l'écart observé dans chaque scénario. Les résultats montrent que le CA calculé est de plus en plus sous-estimé à mesure que la corruption simulée augmente. L'écart entre LOW et CRITICAL est de 6 425 982,22 €, mais c'est bien ORIGINAL qui constitue la référence pour quantifier l'écart global.

4.2 Paiements non rattachés et CA sans client connu

Indicateur	LOW	MEDIUM	CRITICAL
Paiements sans commande (nombre)	1 255	6 275	12 550
Paiements sans commande (montant)	667 860,88 €	3 360 760,84 €	6 459 898,61 €
CA des commandes sans client connu	666 744,37 €	3 397 083,53 €	6 478 206,85 €

Les paiements sans commande sont non attribuables dans le modèle, mais ils ne sont pas nécessairement impayés ou perdus. Quant au CA des commandes sans client connu, il limite les analyses de fidélité et de segmentation CRM.

4.3 Définition du KPI

La somme des lignes de vente diffère légèrement de celle des commandes : 71 312 375,96 € en LOW, 68 456 582,88 € en MEDIUM et 64 812 612,80 € en CRITICAL.
Tout rapport doit donc préciser la définition du KPI utilisée et ne pas mélanger ces deux mesures sans rapprochement préalable.

5. Contrôles et validation

Plusieurs familles de contrôles ont été exécutées, avec des résultats concordants :

- neuf comparaisons de volumes entre CLEAN et DW : toutes validées ;
- neuf comparaisons de sommes entre CLEAN et DW : toutes validées, avec des différences nulles ;
- cinq vérifications des scénarios dans les relations : aucun mélange constaté ;
- dix tests finaux automatisés : tous validés.

Ces dix tests vérifient les volumes, la présence des clés, le rattachement des scénarios et les anomalies attendues. Ils ne démontrent pas pour autant que chaque donnée source est économiquement correcte.

6. Optimisation SQL

Des index ont été ajoutés sur les colonnes de jointure et de filtrage : customer_key, store_key, le couple (scenario, date_key), product_key, ainsi que order_key pour les ventes et les paiements. Les statistiques ont ensuite été actualisées avec ANALYZE.

Les mesures EXPLAIN (ANALYZE, BUFFERS) réalisées après indexation donnent les temps suivants : PostgreSQL a recouru à des parcours séquentiels parallèles, des jointures par hachage et une agrégation parallèle, ce qui est cohérent avec des requêtes qui lisent une grande fraction des tables.

Requête	Temps d'exécution
CA mensuel	174,649 ms
CA par catégorie	344,916 ms
Commandes CRITICAL datées	93,983 ms

Aucun test de performance n'ayant été mené avant l'indexation, ces résultats ne permettent pas d'affirmer un gain de vitesse.

7. Difficultés rencontrées et enseignements

7.1 Difficultés et corrections

Difficulté	Cause et correction
Erreur PowerShell / psql (token « ï»¿SELECT »)	Fichiers encodés avec BOM UTF-8 ; correction en UTF-8 sans BOM
Pager psql (« -- Suite -- »)	Sortie paginée ; quitter avec q ou désactiver le pager
Messages VS Code autour de LOOP / PLpgSQL	Analyseur SQL Server inadapté ; le code s'exécutait correctement dans DBeaver (PostgreSQL)
Alerte Pylance sur cursor.fetchone()	Résultat possiblement None ; vérification avant indexation
Préparation V1 incorrecte	Utilisation de processed_v2 après audit des colonnes
Récupération des références trop longue	Séparation entre le besoin immédiat (mesurer) et le chantier ultérieur (restaurer)
Git : « pathspec sql/04_dw »	Dossier réel sql/04_warehouse ; correction puis push du commit 630d3d6
Avertissement LF/CRLF	Simple avertissement de fin de ligne, sans échec

7.2 Enseignements

Le projet conduit à six règles de travail : vérifier les chemins et les schémas avant toute exécution, distinguer les erreurs d'éditeur des erreurs SQL, contrôler les volumes après chaque import, conserver le lignage, ne jamais modifier les données sans plan de retour arrière, et valider chaque jalon par un commit.

8. Démarche recommandée en mission

En contexte professionnel, la démarche suivrait les étapes ci-dessous :

1. Cadrer le KPI et identifier son propriétaire métier.
2. Inventorier les sources, les contrats de schéma et les règles de qualité.
3. Établir une situation de référence avant toute correction.
4. Mesurer cinq dimensions de qualité : complétude, unicité, validité, cohérence et traçabilité.
5. Classer les anomalies par niveau de gravité.
6. Choisir, pour chacune, entre une correction vérifiée, une exclusion justifiée ou un classement en « Inconnu ».
7. Construire les dimensions et les faits avec une granularité explicite.
8. Valider avec les équipes métier et la finance.
9. Tracer les transformations et tester les régressions.
10. Publier les limites avec chaque tableau de bord et suivre les indicateurs de qualité dans le temps.

Avant de démarrer, plusieurs questions doivent être posées au client : quel système fait foi ; si les montants incluent remises, taxes et retours ; quelle date définit le mois de vente ; comment traiter les paiements non attribués ; à partir de quel seuil d'anomalie un KPI devient inutilisable ; qui approuve une correction ; quel est le coût opérationnel d'une mauvaise décision ; et quels contrôles doivent bloquer le pipeline.

9. Recommandations

Les recommandations sont classées par ordre de priorité :

1. Publier un tableau de bord de qualité présentant, par scénario, les taux de dates manquantes, de références invalides et de CA non attribuable.
2. Documenter une définition canonique du chiffre d'affaires, du panier moyen et des paiements.
3. Étudier une restauration supervisée à partir d'une source de vérité, avec un audit des valeurs avant et après correction.
4. Ajouter des tests de non-régression et une comparaison des performances avant et après indexation.
5. Préparer l'intégration BI sans mélanger les scénarios.

10. Limites

Ce travail présente plusieurs limites : les données sont fictives, la corruption est simulée, la restauration est différée, aucun gain de performance n'est démontré et aucune validation comptable réelle n'a eu lieu.

Les résultats chiffrent la sensibilité des indicateurs à la dégradation des données ; ils n'établissent pas une causalité observée dans une entreprise réelle.

11. Sources textuelles et éthique

NexaRetail est une source transactionnelle synthétique.
Le corpus SetFit/amazon_reviews_multi_fr est indépendant : ne représente pas des avis authentiques de clients NexaRetail.

Trois tentatives d'enrichissement des réclamations ont atteint des taux de couverture de 40,28 %, 18,50 % et 10,15 %.

Elles ont été abandonnées faute de cohérence sémantique garantie, afin de ne pas fabriquer d'associations client-produit invérifiables.

12. Livrables et traçabilité

Le code est disponible dans le dépôt GitHub NacPKZ/ConsultAI-360 (https://github.com/NacPKZ/ConsultAI-360), organisé en cinq dossiers de scripts : sql/03_clean, sql/04_warehouse, sql/05_analytics, sql/06_optimization et sql/07_tests.

Trois commits jalonnent le travail : f1245a3 (CLEAN), 12f515c (analyses, optimisation et tests) et 630d3d6 (entrepôt de données).

Le fichier README du dépôt décrit l'architecture, les KPI, les limites et la procédure de reproduction.

Conclusion

La phase 4 fournit une base analytique vérifiée pour mesurer et expliquer les effets des anomalies de données sur les indicateurs.
Sa valeur de conseil repose sur trois principes : la transparence des métriques, le maintien du lignage, et la distinction claire entre un résultat calculé, la qualité de la donnée et une décision métier.

Annexe : notions SQL clés et questions de maîtrise

A. Lecture du code

Instruction	Rôle dans le projet
CREATE TABLE	Définit le schéma et les types
PRIMARY KEY	Identifie une ligne de façon unique
UNIQUE (scenario, id)	Impose l'unicité métier au sein d'une variante
FOREIGN KEY	Garantit l'existence de la dimension référencée
INSERT … SELECT	Charge les données dans la table cible
ON CONFLICT DO NOTHING	Évite les doublons au rechargement, sans remplacer la validation des changements
LEFT JOIN	Conserve les lignes sans correspondance
COALESCE	Affecte la clé « Inconnu » quand la référence ne correspond à aucune dimension
GROUP BY	Agrège les KPI à une granularité définie
FILTER	Compte les anomalies à l'intérieur d'une agrégation
NULLIF	Évite les divisions par zéro
BEGIN / COMMIT / ROLLBACK	Encadre le chargement dans une transaction annulable en cas d'échec
EXPLAIN ANALYZE	Exécute la requête et expose son plan, ses temps et ses lectures